

TP6 — FNBC y desnormalización controlada

Parte 1 — Normalización del control de lotes

Se analizó $R(L,D,C)$, donde L representa lote, D depósito y C responsable de control, con las dependencias de dominio $LD \rightarrow C$ y $C \rightarrow D$. Cada responsable pertenece a un único depósito; un depósito puede tener varios responsables. No se infirieron dependencias por coincidencias de la muestra.

Las claves candidatas son **LD** y **LC**: ambas determinan todos los atributos y son mínimas, porque $L^+ = L$, $D^+ = D$ y $C^+ = CD$. Son las únicas claves candidatas, ya que ninguna dependencia permite obtener L y toda clave debe incluirlo. **Todos los atributos son primos.**

La relación cumple **3FN**, porque en $C \rightarrow D$ el atributo dependiente D es primo, pero viola **FNBC**: C no es superclave. La repetición del depósito del responsable origina anomalías al registrar una pertenencia sin lote, borrar su último control o actualizar solo algunas de sus filas.

Se descompuso en:

- **responsable_deposito(C,D)**, con PK C.
- **control_lote(L,C)**, con PK LC.

Ambas relaciones quedan en FNBC. La reunión es **sin pérdida** porque el atributo común C determina toda responsable_deposito: $C \rightarrow CD$.

Sin embargo, $LD \rightarrow C$ **no se preserva mediante las restricciones locales**. Si responsable_deposito contiene (801,30) y (803,30), y control_lote contiene (501,801) y (501,803), las PK y FK se cumplen, pero la reunión asigna dos responsables al mismo lote y depósito. Evitarlo requiere un control entre tablas, no implementado en esta parte. Reunión sin pérdida y preservación de dependencias son propiedades distintas.

El script tp_fnbc_control_lote.sql migró mediante INSERT ... SELECT DISTINCT y reconstruyó la relación con v_control_lote_almacen. La evidencia evidencia_fnbc_20261008_222128.txt registra diferencias vacías con EXCEPT bidireccional, conteos **3/2/3/3** y reconstrucción exacta de (501,30,801), (502,30,801) y (503,31,802).

Parte 2 — Top diario de categorías

Adaptación y patrón elegido

La consulta conserva `SUM(dp.subtotal)`, agrupación por nombre y `LIMIT 5`, usando `dp.id_pedido`, `dp.id_producto`, `pr.id_categoria` y un intervalo semiabierto sobre `ped.fecha_hora`. Se mide el **2026-06-25 en America/Buenos_Aires**, sustituyendo `CURRENT_DATE` porque la carga no contiene pedidos de hoy. Se omiten los filtros eliminado, inexistentes en pedido y detalle_pedido, sin agregar filtros por estado o activo.

La copia contiene **200005 pedidos, 499571 detalles, 50003 productos y 2 categorías**. El resultado tiene como máximo dos filas; no representa un top cinco entre muchas categorías.

Se eligieron columnas precalculadas con disparadores porque la medición inicial de **47.151 ms** mostró un recorrido paralelo de pedido y 1539 búsquedas en producto, con 4617 buffers; copiar al detalle la fecha y la categoría permite evitar esos recorridos y sincronizar los cambios dentro de la misma transacción, sin esperar un refresco periódico. Una vista materializada actualizada de noche no satisface el requisito de actualización frecuente. La adaptación es reversible porque conserva las fuentes originales y permite retirar únicamente los objetos añadidos.

Implementación y rendimiento

`tp_desnormalizacion_top_categorias.sql` agrega `fecha_hora_pedido_cache` e `id_categoria_cache`, sus restricciones y el índice `idx_tp6_detalle_fecha_cache`. Un trigger recalcula ambas copias al insertar o actualizar detalles; otros propagan cambios de fecha del pedido y categoría del producto. El nombre se obtiene mediante JOIN con categoría y no se copia.

La medición inicial está en `evidencia_top_antes_20261008_225517.txt`, obtenida con `medir_top_categorias_antes.sql`. La comparación directa está en `evidencia_top_desnormalizacion_20261008_232126.txt`:

Medición	Normalizada (ms)	Desnormalizada (ms)
Ronda 1	42.807	1.855
Ronda 2	41.920	1.815
Promedio	42.3635	1.835

Antes: normalizada	Después: desnormalizada
Promedio: 42.3635 ms	Promedio: 1.835 ms
shared hit: 10942	shared hit: 564
Parallel Seq Scan de pedido y Nested Loop con búsquedas en detalle/producto	Bitmap Heap Scan de detalle mediante el índice de fecha y Nested Loop con categoría

La reducción observada del tiempo es **95.67 %**, con un cociente de tiempos de **23.09**, atribuible al **conjunto columnas más índice**. No se usaron los 47.151 ms iniciales para calcularla. Los buffers shared hit son accesos resueltos en caché, no lecturas físicas ni páginas únicas.

La consulta normalizada recorre pedido y busca detalles y productos. La desnormalizada accede por fecha a los 1539 detalles visibles y conserva únicamente el JOIN con categoría.

Plan normalizado — ronda 1

```
55
56 Ronda 1 – consulta normalizada
57
58
59 Limit (cost=8634.10..8634.10 rows=2 width=39) (actual time=37.926..42.723 rows=2 loops=1)
60   Buffers: shared hit=10942
61   -> Sort (cost=8634.10..8634.10 rows=2 width=39) (actual time=37.925..42.721 rows=2 loops=1)
62     Sort Key: (sum(dp.subtotal)) DESC
63     Sort Method: quicksort Memory: 25kB
64     Buffers: shared hit=10942
65     -> Finalize GroupAggregate (cost=8628.44..8634.09 rows=2 width=39) (actual time=37.915..42.712 rows=2 loops=1)
66       Group Key: c.nombre
67       Buffers: shared hit=10942
68       -> Gather Merge (cost=8628.44..8634.05 rows=2 width=39) (actual time=37.717..42.699 rows=2 loops=1)
69         Workers Planned: 1
70         Workers Launched: 1
71         Buffers: shared hit=10942
72         -> Partial GroupAggregate (cost=7628.43..7633.81 rows=2 width=39) (actual time=10.849..10.945 rows=1 loops=2)
73           Group Key: c.nombre
74           Buffers: shared hit=10942
75           -> Sort (cost=7628.43..7630.22 rows=714 width=14) (actual time=10.768..10.811 rows=770 loops=2)
76             Sort Key: c.nombre
77             Sort Method: quicksort Memory: 98kB
78             Buffers: shared hit=10942
79             Worker 0: Sort Method: quicksort Memory: 25kB
80             -> Hash Join (cost=1.76..7594.59 rows=714 width=14) (actual time=0.028..10.562 rows=770 loops=2)
81               Hash Cond: (pr.id_categoria = c.id)
82               Buffers: shared hit=10934
83               -> Nested Loop (cost=0.71..7587.70 rows=714 width=15) (actual time=0.016..10.346 rows=770 loops=2)
84                 Buffers: shared hit=10933
85                 -> Nested Loop (cost=0.42..7362.82 rows=714 width=15) (actual time=0.013..8.317 rows=770 loops=2)
86                   Buffers: shared hit=6316
87                   -> Parallel Seq Scan on pedido ped (cost=0.00..3236.75 rows=286 width=8) (actual time=0.006..6.374 rows=294 loops=2)
88                     Filter: ((fecha_hora >= '2026-06-25 00:00:00-03'::timestamp with time zone) AND (fecha_hora < '2026-06-26 00:00:00-03'::timestamp with time zone))
89                     Rows Removed by Filter: 99708
90                     Buffers: shared hit=1472
91                   -> Index Scan using pk_detalle_pedido on detalle_pedido dp (cost=0.42..14.40 rows=3 width=23) (actual time=0.005..0.006 rows=3 loops=588)
92                     Index Cond: (id_pedido = ped.id)
93                     Buffers: shared hit=4844
94                   -> Index Scan using producto_pkey on producto pr (cost=0.29..0.31 rows=1 width=16) (actual time=0.002..0.002 rows=1 loops=1539)
95                     Index Cond: (id = dp.id_producto)
96                     Buffers: shared hit=4617
97                 -> Hash (cost=1.02..1.02 rows=2 width=15) (actual time=0.015..0.016 rows=2 loops=1)
98                   Buckets: 1024 Batches: 1 Memory Usage: 9kB
99                   Buffers: shared hit=1
100                 -> Seq Scan on categoria c (cost=0.00..1.02 rows=2 width=15) (actual time=0.012..0.013 rows=2 loops=1)
101                   Buffers: shared hit=1
102 Planning:
103   Buffers: shared hit=43
104 Planning Time: 0.744 ms
105 Execution Time: 42.807 ms
106 (47 rows)
107
```

Plan desnormalizado — ronda 1

108	Ronda 1 – consulta desnormalizada	
109		QUERY PLAN
110		-----
111	Limit (cost=4266.42..4266.43 rows=2 width=39) (actual time=1.805..1.806 rows=2 loops=1)	
112	Buffers: shared hit=564	
113	-> Sort (cost=4266.42..4266.43 rows=2 width=39) (actual time=1.804..1.805 rows=2 loops=1)	
114	Sort Key: (sum(dp.subtotal)) DESC	
115	Sort Method: quicksort Memory: 25kB	
116	Buffers: shared hit=564	
117	-> GroupAggregate (cost=28.97..4266.41 rows=2 width=39) (actual time=1.523..1.798 rows=2 loops=1)	
118	Group Key: c.nombre	
119	Buffers: shared hit=564	
120	-> Nested Loop (cost=28.97..4258.38 rows=1602 width=14) (actual time=0.301..1.577 rows=1539 loops=1)	
121	Join Filter: (c.id = dp.id_categoria_cache)	
122	Rows Removed by Join Filter: 1539	
123	Buffers: shared hit=564	
124	-> Index Scan using categoria_nombre_key on categoria c (cost=0.13..12.16 rows=2 width=15) (actual time=0.006..0.008 rows=2 loops=1)	
125	Buffers: shared hit=2	
126	-> Materialize (cost=28.84..4202.17 rows=1602 width=15) (actual time=0.146..0.643 rows=1539 loops=2)	
127	Buffers: shared hit=562	
128	-> Bitmap Heap Scan on detalle_pedido dp (cost=28.84..4194.16 rows=1602 width=15) (actual time=0.290..0.999 rows=1539 loops=1)	
129	Recheck Cond: ((fecha_hora_pedido_cache >= '2026-06-25 00:00:00-03'::timestamp with time zone) AND (fecha_hora_pedido_cache < '2026-06-26 00:00:00-03'::timestamp with time zone))	
130	Heap Blocks: exact=558	
131	Buffers: shared hit=562	
132	-> Bitmap Index Scan on idx_tp6_detalle_fecha_cache (cost=0.00..28.44 rows=1602 width=0) (actual time=0.170..0.170 rows=1542 loops=1)	
133	Index Cond: ((fecha_hora_pedido_cache >= '2026-06-25 00:00:00-03'::timestamp with time zone) AND (fecha_hora_pedido_cache < '2026-06-26 00:00:00-03'::timestamp with time zone))	
134	Buffers: shared hit=4	
135	Planning:	
136	Buffers: shared hit=8	
137	Planning Time: 0.431 ms	
138	Execution Time: 1.855 ms	
139	(28 rows)	
140		
141	Ronda 2 – consulta desnormalizada	
142		QUERY PLAN
143		-----

Validación y límites

Ambas consultas devolvieron **Bebidas: 5589534.40** y **Pizzas: 5215387.22**. El EXCEPT bidireccional y la auditoría completa de las columnas redundantes devolvieron cero filas.

Las pruebas secuenciales verificaron INSERT, UPDATE, cambios de padres, propagación de fecha y categoría, cambio de nombre, DELETE y CASCADE. Usaron IDs explícitos sin consumir secuencias y se deshicieron con SAVEPOINT antes de medir.

Auditoría y pruebas de sincronización

```
25 SAVEPOINT
26 psql:TP6_FNBC_Desnormalizacion/tp_desnormalizacion_top_categorias.sql:713: NOTICE:  Pruebas de sincronización secuenciales superadas; no son pruebas concurrentes.
27 DO
28 ROLLBACK
29 RELEASE
30 | id_pedido | id_producto | fecha_hora_pedido_cache | fecha_hora_original | id_categoria_cache | id_categoria_original
31 -----+-----+-----+-----+-----+-----
32 (0 rows)
33
34 | sentido | categoria | total_vendido
35 -----+-----+-----
36 (0 rows)
37
38 psql:TP6_FNBC_Desnormalizacion/tp_desnormalizacion_top_categorias.sql:745: NOTICE:  Auditoría completa y EXCEPT bidireccional: resultados vacíos.
39 DO
```

La sincronización soporta **READ COMMITTED**, utiliza bloqueos FOR SHARE sobre los padres y puede generar interbloqueos que requieran reintentar la transacción completa. **La concurrencia entre sesiones no fue probada.**

El cambio aumenta el tamaño de las filas y el trabajo de escritura, mantenimiento del índice y propagación a múltiples detalles; ese costo no fue cuantificado. Las mediciones se realizaron después de cargar e indexar, dentro de la misma transacción, con ANALYZE, calentamiento previo y orden alternado. Los bloqueos ACCESS EXCLUSIVE impidieron actividad concurrente; no hubo VACUUM y dos rondas no constituyen un benchmark exhaustivo.

Alcance y referencias

Los fundamentos y resultados detallados están en informe_parte1_fnbc.md e informe_parte2_desnormalizacion.md, dentro de TP6_FNBC_Desnormalizacion.

Ambos scripts de implementación terminan con ROLLBACK, confirmado en sus evidencias: ni la migración de Parte 1 ni la instalación de Parte 2 quedaron persistidas.